# Projet R : Régression Linéaire, stepAIC, PCR, PLS, Ridge et Lasso

## Objectif du Projet

On souhaite modéliser l'indice d'octane moteur de l'essence en fonction de son processus de fabrication impliquant plusieurs composantes. On dispose des variables suivantes :

- y = indice octane moteur
- x1 = distilation directe (valeur comprise entre 0 et 0.21)
- x2 = reformat (valeurs entre 0 et 0.62)
- x3 = naphta de craquage termique (entre 0 et 0.12)
- x4 = naphta de craquage cataitique (entre 0 et 0.62)
- x5 = polymere (entre 0 et 0.12)
- x6 = alkylat (entre 0 et 0.74)
- x7 = essence naturelle (entre 0 et 0.08)

Pour cela on va faire une analyse de régression linéaire et on va choisir le meilleur modèle prédictif.

## Avant tout

Installer les packages pls, glmnet et jetez un coup d'oeil sur les fonctions *pcr, plsr, glmnet :*

```r
install.packages("pls") # installe le package pls
library(pls)            # charge la librairie pls
help(package = "pls")   # donnee des informations sur pls

install.packages("glmnet")
library(glmnet)
help(package ="glmnet")
```

## Création du Dataframe

```r
essence <- data.frame(
  x1 = c(0, 0, 0, 0, 0, 0, 0.17, 0.17, 0.17, 0.17, 0.21, 0),
  x2 = c(0.23, 0.10, 0.00, 0.49, 0, 0.62, 0.27, 0.19, 0.21, 0.15, 0.36, 0),
  x3 = c(0, 0, 0, 0, 0, 0, 0.10, 0.10, 0.10, 0.10, 0.12, 0),
  x4 = c(0, 0, 0.10, 0, 0.62, 0, 0.38, 0.38, 0.38, 0.38, 0.25, 0.55),
  x5 = c(0, 0.12, 0.12, 0.12, 0.12, 0, 0, 0.02, 0, 0.02, 0, 0),
  x6 = c(0.74, 0.74, 0.74, 0.37, 0.18, 0.37, 0, 0.06, 0.06, 0.10, 0, 0.37),
  x7 = c(0.03, 0.04, 0.04, 0.02, 0.08, 0.01, 0.08, 0.08, 0.08, 0.08, 0.06, 0.08),
  y  = c(98.7, 97.8, 96.6, 92.0, 86.6, 91.2, 81.9, 83.1, 82.4, 83.2, 81.4, 88.1)
)

write.csv(essence, "octane_model.csv", row.names = FALSE)

essence <- read.csv("octane_model.csv")
```

Le dataframe “essence” est crée et il est sauvegardé en fichier “octane_model.csv”.
On peut le rouvrir à chaque fois avec read.csv().

## Affichage des statistiques de base

```
> summary(essence)
       x1                x2                x3                x4                x5          
 Min.   :0.00000   Min.   :0.0000   Min.   :0.00000   Min.   :0.0000   Min.   :0.00000  
 1st Qu.:0.00000   1st Qu.:0.0750   1st Qu.:0.00000   1st Qu.:0.0000   1st Qu.:0.00000  
 Median :0.00000   Median :0.2000   Median :0.00000   Median :0.3150   Median :0.01000  
 Mean   :0.07417   Mean   :0.2183   Mean   :0.04333   Mean   :0.2533   Mean   :0.04333  
 3rd Qu.:0.17000   3rd Qu.:0.2925   3rd Qu.:0.10000   3rd Qu.:0.3800   3rd Qu.:0.12000  
 Max.   :0.21000   Max.   :0.6200   Max.   :0.12000   Max.   :0.6200   Max.   :0.12000  
       x6                x7                y         
 Min.   :0.0000   Min.   :0.01000   Min.   :81.40  
 1st Qu.:0.0600   1st Qu.:0.03750   1st Qu.:82.92  
 Median :0.2750   Median :0.07000   Median :87.35  
 Mean   :0.3108   Mean   :0.05667   Mean   :88.58  
 3rd Qu.:0.4625   3rd Qu.:0.08000   3rd Qu.:93.15  
 Max.   :0.7400   Max.   :0.08000   Max.   :98.70  
```

## Expliquer/prédire l’indice d’octane moteur de l’essence en fonction de son processus de fabrication

### Régression linéaire “oridinaire” :

```
> modele <- lm(y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7, data = essence)
> summary(modele)

Call:
lm(formula = y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7, data = essence)

Residuals:
         1          2          3          4          5          6          7          8 
 1.207e+00 -2.218e-01 -5.475e-01 -8.195e-02  8.105e-01 -3.527e-01  5.906e-02  3.598e-01 
         9         10         11         12 
-3.036e-01 -1.153e-01  1.055e-15 -8.141e-01 

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   116.92      81.10   1.442    0.209
x1            -82.60     173.22  -0.477    0.654
x2            -31.00      80.84  -0.383    0.717
x3             24.33     431.97   0.056    0.957
x4            -39.74      90.25  -0.440    0.678
x5            -29.17      84.02  -0.347    0.743
x6            -16.62      84.45  -0.197    0.852
x7                NA         NA      NA       NA

Residual standard error: 0.8362 on 5 degrees of freedom
Multiple R-squared:  0.9925,    Adjusted R-squared:  0.9836 
F-statistic: 110.7 on 6 and 5 DF,  p-value: 3.762e-05
```

Malgré un bon $R^2 = 98\%$ et un test global de Fisher nous indiquant que le modèle explique l'indice d'octane de manière significative (p-value = 3.762e-05 << 0.05), les **p-values individuelles** sont toutes > 0.05 → cela veut dire qu'aucune variable prise isolément n'est significative, et la variable **x7** est non défini (*NA*).

L'explication peut venir d'une inflation de la variance des estimateurs due soit au fait que le nombre d'observations est plus petit que le nombre de variables (ce n'est pas le cas ici), soit au fait que la matrice ($X^T X$) est mal-conditionnée (son inverse n'existe pas ou a des valeurs trop grandes) et dans ce dernier cas on parle de *multi-colinéarité* : une ou plusieurs variables prédicteurs sont redondantes dans l'analyse.

En termes de pouvoir prédictif, ce modèle fournit une erreur de prédiction par validation croisée (leave-one-out) égale à MSEP (Mean Squares Error of Prediction) :

```
> y_pred_complet <- predict(modele, essence) # Calcul du RMSEP
> rmsep <- sqrt(mean((essence$y - y_pred_complet)^2))
> rmsep
[1] 0.5397588
>
```

## Diagnostiquer la multi-colinéarité

On réalise une **ACP** afin d'identifier des valeurs propres proches de 0 :

```
> library(FactoMineR)
> acp = PCA(essence[,-8], scale=TRUE, graph = FALSE) #-8 : sans la variable y
> print(acp$eig)
        eigenvalue percentage of variance cumulative percentage of variance
comp 1 4.025525e+00          5.750751e+01                          57.50751
comp 2 1.910353e+00          2.729076e+01                          84.79827
comp 3 5.726286e-01          8.180409e+00                          92.97868
comp 4 4.781750e-01          6.831072e+00                          99.80975
comp 5 1.323651e-02          1.890930e-01                          99.99884
comp 6 8.113963e-05          1.159138e-03                         100.00000
comp 7 2.319397e-32          3.313424e-31                         100.00000
```

- **Composante 1 (57,5 % de variance)** : elle résume plus de la moitié de l'information contenue dans tes 7 variables.
- **Composante 2 (27,3 % de variance)** : elle ajoute encore près d'un tiers de l'information.
- Ensemble, **les deux premières composantes expliquent ~85 % de la variance totale** → c'est excellent,
- on peux travailler avec seulement ces deux dimensions au lieu des 7 variables initiales.
- Les composantes suivantes (3, 4) n'apportent que peu d'information supplémentaire.

➢ Effectivement, les trois dernières valeurs propres sont particulièrement proches de 0 et elles expliquent quantités négligeables d'information. Multi-colinéarité en vue!

Une autre manière de diagnostiquer la multicolinéarité et d'investiguer la statistique VIF (Variance Inflation Factor) qui pour chaque variable prédicteur $X_i \in \{X_1, \ldots, X_p\}$ calcule la quantité

$$VIF(X_i) = \frac{1}{1 - R^2}$$

où $R_i^2$ est le $R^2$ de la variable $X_i$ par les autres variables $\{X_1, \ldots, X_{i-1}, X_{i+1}, \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots, X_p\}$.

Une valeur de VIF supérieure à 10 évoque une quasi-certitude sur la présence de multi-colinéarité

```
> modele2 <- lm(y ~ x1 + x2 + x3 + x4 + x5 + x6, data = essence)
> library(car)
> vif(modele2)
       x1        x2        x3        x4        x5        x6
4020.4379 3930.6304 8504.3044 6602.7757  362.0785 9625.9219
>
```

VIF très élevés (> 300, certains > 9000), confirmant une forte redondance entre les variables.

## Solutions à la multi-colinéarité : stepAIC, PCR, PLS, Ridge et Lasso

### stepAIC

C'est la solution la plus simple : choisir un modèle de taille plus petite qui inclut juste les variables qui ajustent le mieux les données en termes de vraisemblance pénalisée :

```
> library(MASS)
> m_aic=stepAIC(modele, trace=0) # pas d'affichage du processus de selection
> summary(m_aic)

Call:
lm(formula = y ~ x1 + x2 + x4 + x5, data = essence)

Residuals:
    Min      1Q  Median      3Q     Max
-0.8147 -0.2742 -0.1112  0.2329  1.2051

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  100.980      0.749 134.828 3.26e-13 ***
x1           -40.147      2.990 -13.425 2.98e-06 ***
x2           -15.152      1.528  -9.918 2.26e-05 ***
x4           -21.936      1.352 -16.230 8.21e-07 ***
x5           -12.773      4.866  -2.625   0.0341 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.716 on 7 degrees of freedom
Multiple R-squared:  0.9923,    Adjusted R-squared:  0.9879
F-statistic: 226.4 on 4 and 7 DF,  p-value: 1.768e-07
```

- **Variables retenues** : x1, x2, x4, x5.
- **Coefficients** : tous sont significatifs (p-values < 0.05), ce qui veut dire que ces variables expliquent bien l'indice d'octane.
- **$R^2$ ajusté = 0.9879** : le modèle explique environ 99 % de la variance, avec une excellente qualité d'ajustement.
- **Erreur résiduelle standard = 0.716** : les prédictions sont très proches des valeurs observées.
- Les variables x3, x6, x7 ont été éliminées car elles n'apportaient pas d'information supplémentaire (colinéarité ou non-significativité).

En termes de pouvoir prédictif, ce modèle fournit une erreur de prédiction par validation croisée (leave-one-out) égale à MSEP (Mean Squares Error of Prediction) :

```
> y_pred <- predict(m_aic, essence) # Calcul du RMSEP
> rmsep_stepAIC <- sqrt(mean((essence$y - y_pred)^2))
> rmsep_stepAIC
[1] 0.546829
```

- Les deux modèles ont une erreur de prédiction très faible (≈ 0.5), ce qui montre qu'ils prédisent bien l'indice d'octane.
- Le modèle **complet** est légèrement meilleur en RMSEP, mais il souffre de **multicolinéarité** (coefficients instables).
- Le modèle **StepAIC** est un peu moins précis en RMSEP, mais il est **plus simple et plus robuste** car il ne garde que les variables significatives (x1, x2, x4, x5).

**Régression sur les composantes principales (PCR)**

```
> library(pls)

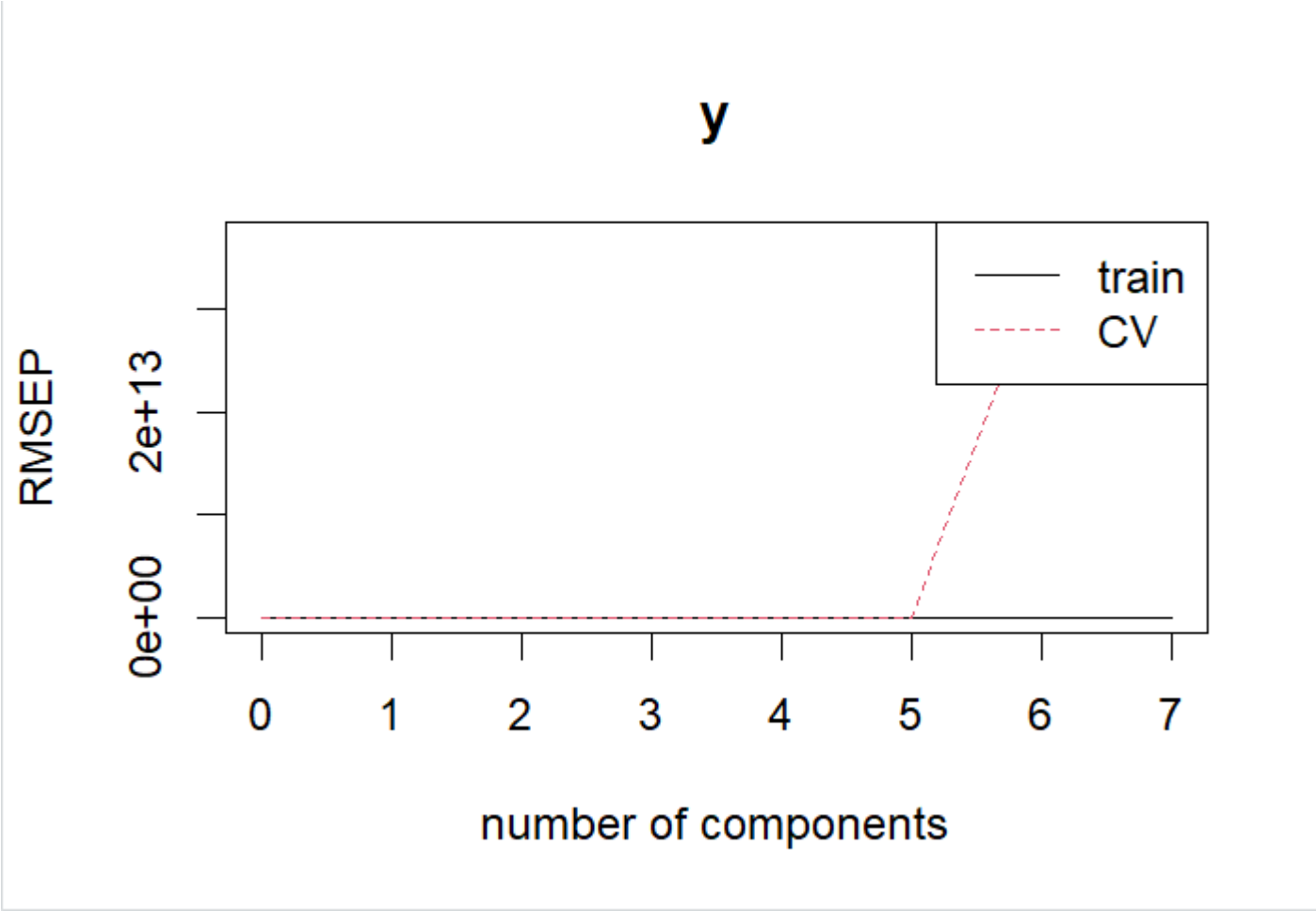
Attachement du package : 'pls'

L'objet suivant est masqué depuis 'package:stats':

    loadings

> m_pcr=pcr(y~., scale=TRUE, validation="LOO", jackknife = TRUE, data=essence)
> plot(m_pcr,"validation", estimate = c("train", "CV"), legendpos = "topright")
```

On va essayer d'évaluer la performance du modèle ACP via la régression PCR.
On a utilisé le package pls avec validation croisée Leave-One-Out.




- **Train (ligne noire)** : erreur sur les données d'entraînement. Elle reste très faible, ce qui est normal car le modèle s'ajuste parfaitement aux données.
- **CV (ligne rouge en pointillés)** : erreur en validation croisée (LOO = Leave-One-Out).
- On observe que le RMSEP reste bas jusqu'à environ **4–5 composantes**, puis explose → signe de **sur-ajustement (overfitting)** si on ajoute trop de composantes.

Le **nombre optimal de composantes** est celui où le RMSEP en validation croisée est minimal, avant qu'il ne remonte brutalement.

Dans ce cas, il semble que **2 à 4 composantes** suffisent pour expliquer la variance et prédire correctement `y`.

```
> summary(m_pcr)
Data:   X dimension: 12 7
        Y dimension: 12 1
Fit method: svdpc
Number of components considered: 7

VALIDATION: RMSEP
Cross-validated using 12 leave-one-out segments.
       (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps    6 comps    7 comps
CV           6.811    2.299    2.537    2.744    1.312    1.431  3.400e+13  3.704e+13
adjCV        6.811    2.278    2.517    2.800    1.280    1.392  3.255e+13  3.546e+13

TRAINING: % variance explained
   1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
X    57.51    84.80    92.98    99.81   100.00   100.00   100.00
y    89.69    90.71    91.87    99.06    99.25    99.25    99.25
```

- **Intercept seul** : RMSEP ≈ 6.81 → très mauvais, car aucune variable explicative.
- **1 composante** : RMSEP ≈ 2.30 → déjà une nette amélioration.
- **2 composantes** : RMSEP ≈ 2.54 → un peu plus élevé.
- **3 composantes** : RMSEP ≈ 2.74 → encore plus élevé.
- **4 composantes** : RMSEP ≈ 1.31 → le meilleur RMSEP obtenu.
- **5 composantes** : RMSEP ≈ 1.43 → proche du minimum, mais pas mieux.
- **6–7 composantes** : RMSEP explose (≈ 10^13) → sur-ajustement / instabilité numérique.
- Donc, le **nombre optimal de composantes** est **4**, car c'est là que le RMSEP est minimal en validation croisée.

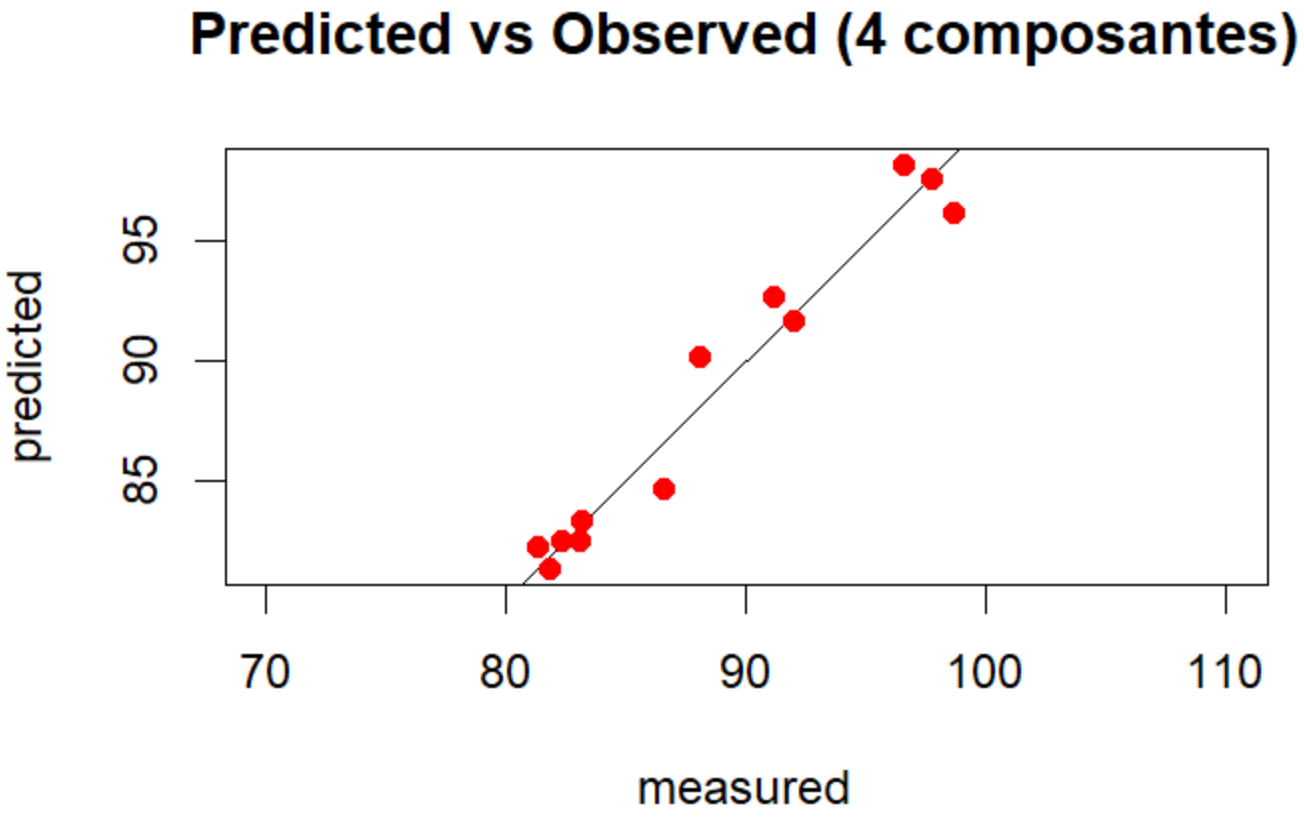
**Comparaison avec les autres modèles**

- **Modèle complet** : RMSEP ≈ 0.54
- **StepAIC** : RMSEP ≈ 0.55
- **PCR (4 composantes)** : RMSEP ≈ 1.31

Cela montre que les modèles linéaires (complet et StepAIC) sont plus précis sur ce petit jeu de données.
Le PCR est un peu moins performant en RMSEP, mais il reste intéressant car il **supprime la multi-colinéarité** et donne un modèle plus stable et généralisable.

Puisque **4 composantes donnent le RMSEP minimal (≈ 1.31)**, il est pertinent de tracer le graphique **observé vs prédit** avec ces 4 composantes. Cela nous permettra de visualiser la qualité des prédictions.

```
> # Graphique Observé vs Prédit avec 4 composantes
> obsfit <- predplot(m_pcr, ncomp = 4, which = "validation",
+                    asp = 1, line = TRUE,
+                    main = "Predicted vs Observed (4 composantes)")
> # Ajouter les points en rouge
> points(obsfit, pch = 16, col = "red")
```



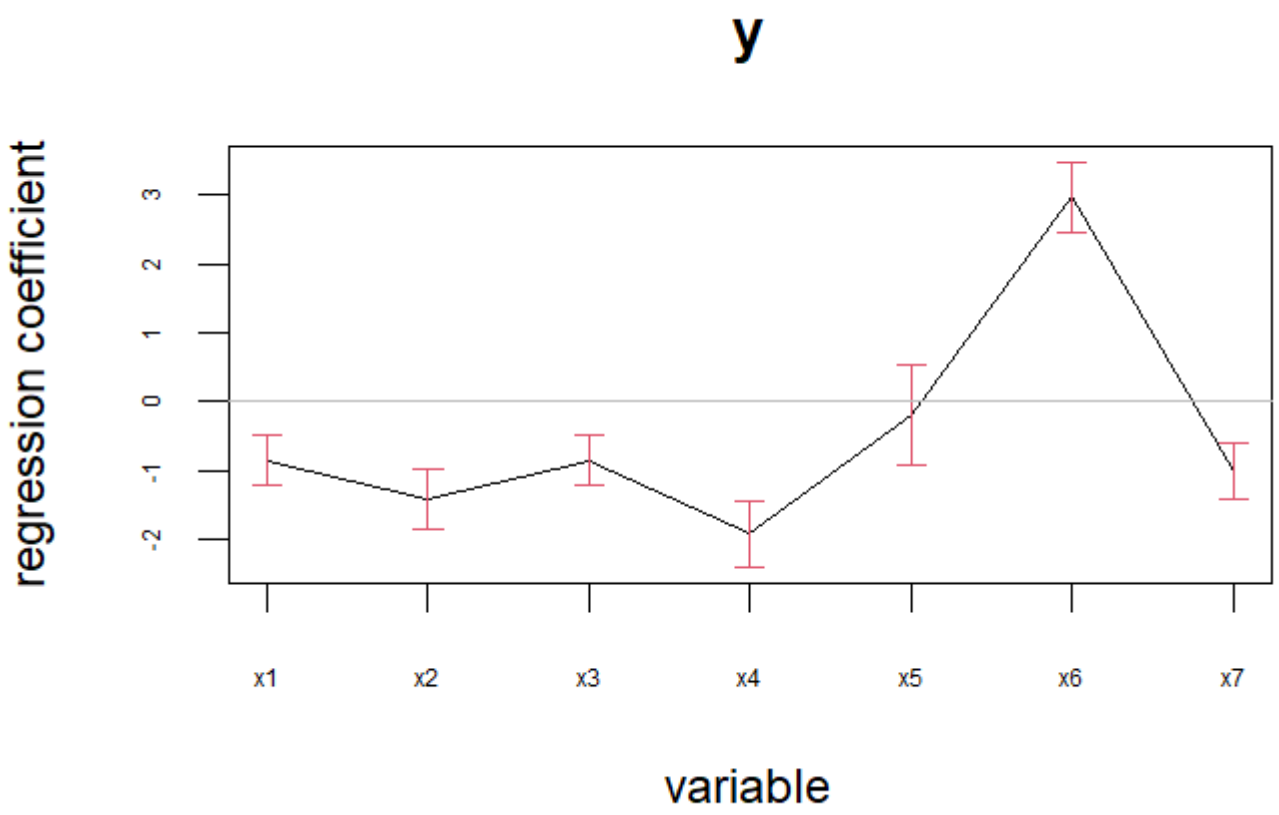

La majorité des points rouges sont proches de la diagonale → le modèle prédit correctement les valeurs observées.

Quelques écarts existent, mais ils restent modérés, cohérents avec le RMSEP ≈ 1.31.

Cela confirme que le modèle PCR avec 4 composantes est **robuste** : il réduit la colinéarité et garde une bonne précision.

Les coefficients et leur significativité est obtenue avec :

```
coefplot(m_pcr, ncomp=4, se.whiskers = TRUE, labels = prednames(m_pcr), cex.axis = 0.5)
```




**Axe des abscisses** : les variables explicatives (x1 à x7).
**Axe des ordonnées** : les coefficients de régression associés à chaque variable dans le modèle PCR avec 4 composantes.
**Barres roses verticales (se.whiskers)** : elles indiquent l'intervalle de confiance autour de chaque coefficient, donc la précision de l'estimation.
**Valeurs positives** : variables qui contribuent positivement à l'indice d'octane.
**Valeurs négatives** : variables qui contribuent négativement.
Par exemple, dans ce graphe x6 a un coefficient positif élevé, cela signifie qu'il augmente la valeur prédite de y. Alors que x4 est fortement négatif, il diminue la valeur prédite.

On va faire un test **jackknife** sur le modèle PCR avec 4 composantes pour avoir une lecture très intéressante des coefficients et de leur significativité :

```
> jack.test(m_pcr, ncomp=4)
Response y (4 comps):
   Estimate Std. Error Df t value  Pr(>|t|)
x1 -0.84851    0.36888 11 -2.3002 0.0420169 *
x2 -1.40576    0.43988 11 -3.1958 0.0085180 **
x3 -0.84840    0.35897 11 -2.3634 0.0375866 *
x4 -1.91092    0.47733 11 -4.0034 0.0020743 **
x5 -0.17922    0.72739 11 -0.2464 0.8099164
x6  2.97125    0.51203 11  5.8029 0.0001188 ***
x7 -1.01182    0.40050 11 -2.5264 0.0281542 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- Le modèle PCR avec 4 composantes met en évidence **x6** comme facteur principal, augmentant fortement l'indice d'octane.
- Les variables **x2, x4, x7** contribuent négativement et de manière significative.
- Les variables **x1 et x3** ont un effet négatif plus modéré.
- **x5** n'a pratiquement aucun impact.
- Globalement, le modèle est cohérent avec le test jackknife : il confirme l'importance de **x6** et de **x4** comme variables clés.

Ces coefficients sont cependant ceux obtenus avec les variables réduites (mais pas centrées), car ACP normée au départ. Les coefficients des variables d'origine sont donc :

```
> # Extraire les coefficients du modèle PCR (variables réduites)
> coef_reduced <- as.vector(coef(m_pcr, ncomp = 4, intercept = TRUE))
> # Calcul des écarts-types et moyennes des variables explicatives (x1 à x7)
> sds <- apply(essence[, 1:7], 2, sd)
> means <- apply(essence[, 1:7], 2, mean)
> # Ajustement des coefficients pour revenir aux variables originales
> coef_original <- coef_reduced
> coef_original[2:8] <- coef_reduced[2:8] / sds
> # Ajustement de l'intercept
> intercept_adj <- coef_reduced[1] - sum(means * (coef_reduced[2:8] / sds))
> coef_original[1] <- intercept_adj
> # Résultat final : intercept + coefficients sur variables originales
> coef_original
[1]  96.917813  -9.193922  -7.189355 -15.762646  -8.417798  -3.138663  10.144250 -37.363887
```

Ces coefficients sont maintenant **directement interprétables** sur les variables brutes (non réduites).

Par exemple : une augmentation de 1 unité de x6 entraîne une hausse d'environ **+10.14** de y.

À l'inverse, une augmentation de 1 unité de x7 entraîne une baisse d'environ **-37.36** de y.

L'intercept (≈ 96.92) correspond à la valeur de y quand toutes les variables sont à leur moyenne.

Pour prédire, on construit un data frame test et on applique la fonction predict et on obtient la prédiction :

```
> # Exemple d'une nouvelle observation
> test_data <- data.frame(
+   x1 = 0.15,
+   x2 = 0.30,
+   x3 = 0.05,
+   x4 = 0.40,
+   x5 = 0.10,
+   x6 = 0.20,
+   x7 = 0.05
+ )
> # Prédiction avec le modèle PCR (4 composantes)
> predict(m_pcr, ncomp = 4, newdata = test_data)
, , 4 comps

         y
1 84.90622
```

**Régression PLS**

```
> library(pls)

Attachement du package : 'pls'

L'objet suivant est masqué depuis 'package:stats':

    loadings
```

```
> #Régression PLS
> m_pls=plsr(y~., scale=TRUE, validation="LOO", jackknife = TRUE, data=essence)
> summary(m_pls)
Data:   X dimension: 12 7
        Y dimension: 12 1
Fit method: kernelpls
Number of components considered: 7

VALIDATION: RMSEP
Cross-validated using 12 leave-one-out segments.
       (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps    6 comps    7 comps
CV           6.811    2.155    1.858    1.300    1.330    1.429  2.728e+12  2.684e+12
adjCV        6.811    2.138    1.813    1.261    1.299    1.391  2.612e+12  2.569e+12

TRAINING: % variance explained
   1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
X    57.36    72.61    91.83    99.75   100.00   100.00   100.40
y    92.36    97.63    99.06    99.08    99.25    99.25    99.25
```

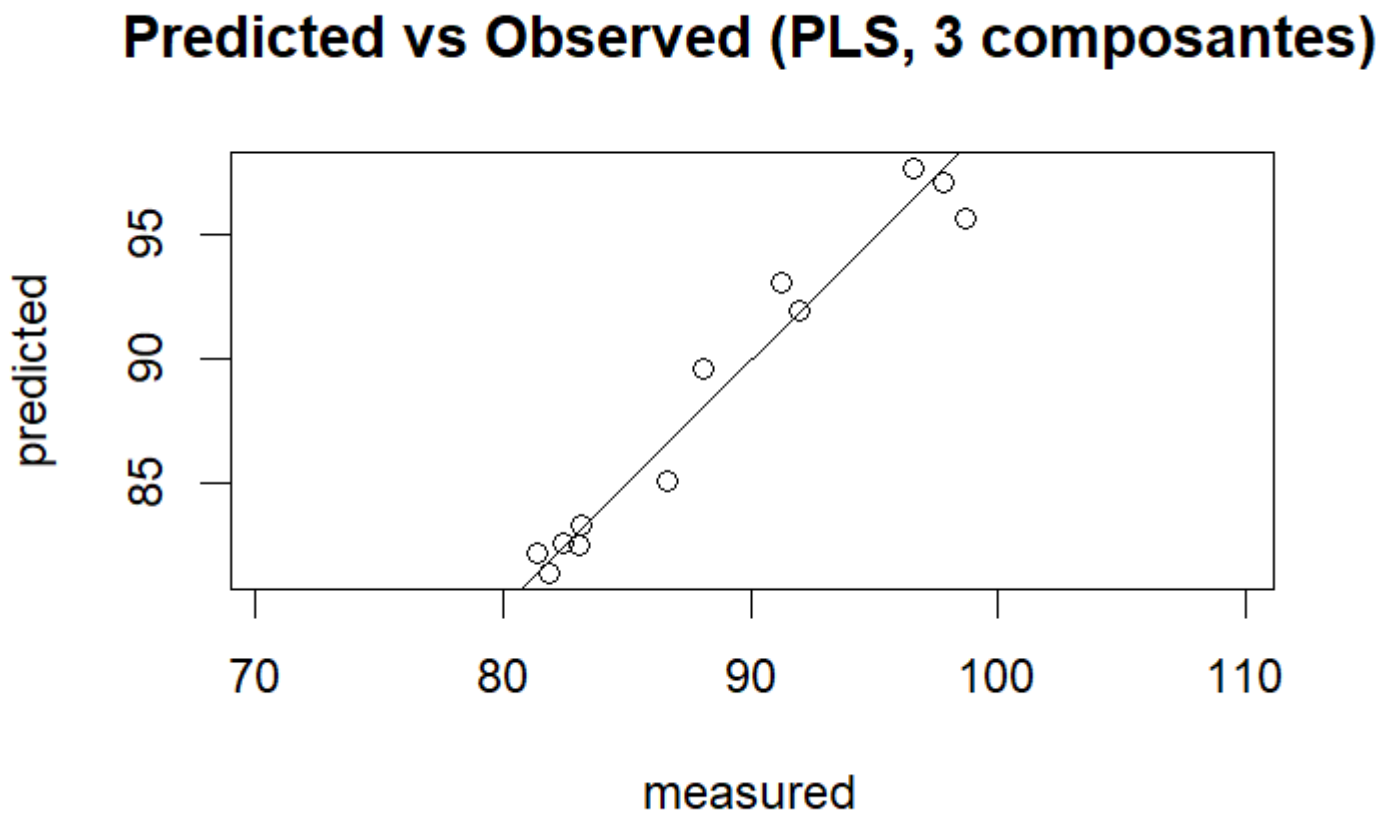
Le modèle PLS optimal est celui avec **3 composantes**, car il minimise le RMSEP.

Ce résultat est cohérent avec la logique de validation croisée : on choisit le nombre de composantes qui donne l'erreur de prédiction la plus faible.

Puisque **3 composantes donnent le RMSEP minimal (≈ 1.30)**, il est pertinent de tracer le graphique **observé vs prédit** avec ces 3 composantes. Cela nous permettra de visualiser la qualité des prédictions.

```
# Graphique Observé vs Prédit avec 3 composantes
predplot(m_pls, ncomp = 3, which = "validation",
         asp = 1, line = TRUE,
         main = "Predicted vs Observed (PLS, 3 composantes)")
```

Voilà le graphe obtenu :



La majorité des points sont proches de la diagonale, ce qui indique une bonne concordance entre les valeurs observées et prédites.
Il y a quelques écarts, mais globalement le modèle capture bien la tendance.
Le RMSEP ≈ 1.30 confirme que l'erreur moyenne de prédiction est faible.
Comparé au modèle PCR (RMSEP ≈ 1.31), le PLS avec 3 composantes est légèrement plus performant.

**On va faire un test jackknife sur le modèle PLS avec 3 composantes pour avoir une lecture très intéressante des coefficients et de leur significativité :**

```
> # Coefficients avec 3 composantes
> coef(m_pls, ncomp = 3, intercept = TRUE)
, , 3 comps

                    y
(Intercept) 92.6759895
x1          -0.9070544
x2          -1.3609484
x3          -0.8970354
x4          -1.9118295
x5          -0.2506157
x6           2.9762416
x7          -0.9350486

> # Test jackknife pour significativité
> jack.test(m_pls, ncomp = 3)
Response y (3 comps):
   Estimate Std. Error Df t value  Pr(>|t|)
x1 -0.90705    0.35211 11 -2.5760 0.0257721 *
x2 -1.36095    0.45710 11 -2.9774 0.0125785 *
x3 -0.89704    0.34849 11 -2.5741 0.0258610 *
x4 -1.91183    0.44516 11 -4.2947 0.0012671 **
x5 -0.25062    0.98577 11 -0.2542 0.8040052
x6  2.97624    0.56357 11  5.2811 0.0002598 ***
x7 -0.93505    0.36254 11 -2.5791 0.0256299 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- Le modèle PLS avec 3 composantes est optimal (RMSEP ≈ 1. 31).
- Les variables clés sont x6 (positif fort) et x4 (négatif fort).
- Les variables x1, x2, x3, x7 sont également significatives mais moins influentes.
- x5 n'apporte pas d'information utile.
- Ce modèle est légèrement plus performant que le PCR (RMSEP ≈ 1.31), ce qui confirme l'intérêt du PLS dans la comparaison finale.

Ces coefficients sont cependant ceux obtenus avec les variables réduites (mais pas centrées), car ACP normée au départ. Les coefficients des variables d'origine sont donc :

```
> # Extraire les coefficients PLS (variables réduites)
> coef_reduced <- as.vector(coef(m_pls, ncomp = 3, intercept = TRUE))
> # Calcul des écarts-types et moyennes des variables explicatives
> sds <- apply(essence[, 1:7], 2, sd)
> means <- apply(essence[, 1:7], 2, mean)
> # Ajustement des coefficients pour revenir aux variables originales
> coef_original <- coef_reduced
> coef_original[2:8] <- coef_reduced[2:8] / sds
> # Ajustement de l'intercept
> intercept_adj <- coef_reduced[1] - sum(means * (coef_reduced[2:8] / sds))
> coef_original[1] <- intercept_adj
> # Résultat final : intercept + coefficients sur variables originales
> coef_original
[1]  96.768646  -9.828318  -6.960181 -16.666239  -8.421802  -4.388934  10.161304 -34.528959
```

Ces coefficients sont maintenant **directement interprétables** sur les variables brutes (non réduites).

Par exemple : une augmentation de 1 unité de x6 entraîne une hausse d'environ **+10.16** de y.

À l'inverse, une augmentation de 1 unité de x7 entraîne une baisse d'environ **-34.52** de y.

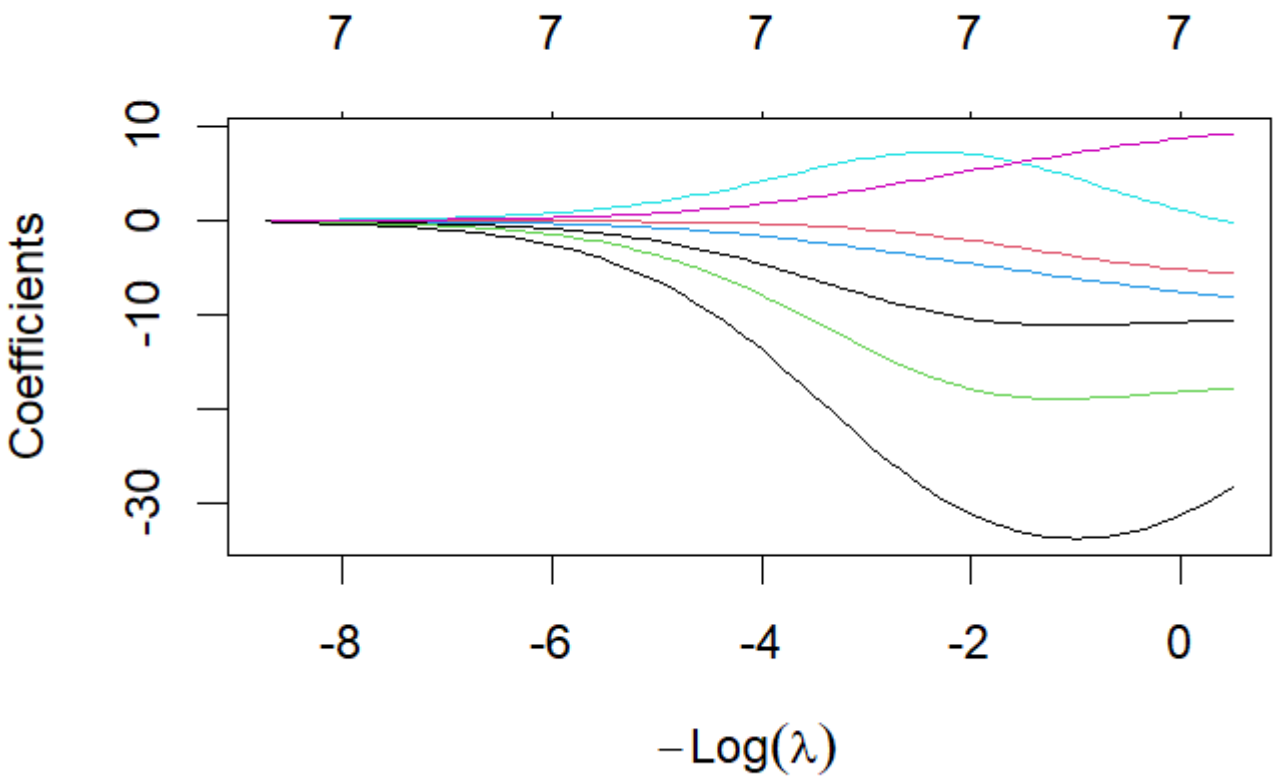
L'intercept (≈ 96.77) correspond à la valeur de y quand toutes les variables sont à leur moyenne.

**Régression Ridge**

On va faire l'ajustement du modèle Ridge en préparant les données pour glmnet :

```
> library(glmnet)
Le chargement a nécessité le package : Matrix
Loaded glmnet 4.1-10
> # Préparation des données
> X <- as.matrix(essence[,1:7])
> y <- essence$y
> # Régression Ridge (alpha=0)
> m_ridge <- glmnet(X, y, alpha = 0)
> # Visualiser les coefficients en fonction de lambda
> plot(m_ridge, xvar = "lambda")
```

La visualisation des coefficients en fonction de lambda :




À gauche (λ grand) : les coefficients sont fortement contraints et proches de zéro → le modèle est très régularisé.

À droite (λ petit) : les coefficients reprennent des valeurs plus proches de celles du modèle complet (non régularisé).

Contrairement au Lasso, aucun coefficient n'est annulé : Ridge conserve toutes les variables, mais réduit leur amplitude.

La régularisation agit comme un « lissage » : elle évite que certains coefficients deviennent trop extrêmes, ce qui limite le sur-ajustement.

Le choix du paramètre lambda (validation croisée) et le calcul de RMSEP :

```
> cv_ridge <- cv.glmnet(X, y, alpha = 0)
Message d'avis :
Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per fold

> 8# Lambda optimal
[1] 8
> lambda_opt <- cv_ridge$lambda.min
> # RMSEP associé
> rmsep_ridge <- sqrt(mean((y - predict(cv_ridge, s=lambda_opt, newx=X))^2))
> rmsep_ridge
[1] 0.6807731
```

- Lambda optimal : ≈ 8
- RMSEP Ridge : ≈ 0.68, ce qui est nettement plus faible que PCR (≈1.31) et PLS (≈1.26).

On peut maintenant extraire les coefficients du modèle Ridge pour voir l'impact de chaque variable :

```
> #Les coefficients du modèle Ridge optimal
> coef_ridge <- coef(cv_ridge, s = "lambda.min")
> coef_ridge
8 x 1 sparse Matrix of class "dgCMatrix"
             lambda.min
(Intercept)  92.1575174
x1          -10.6389664
x2           -5.5543621
x3          -17.7589230
x4           -8.1412179
x5           -0.1400783
x6            9.2166845
x7          -28.2213975
```

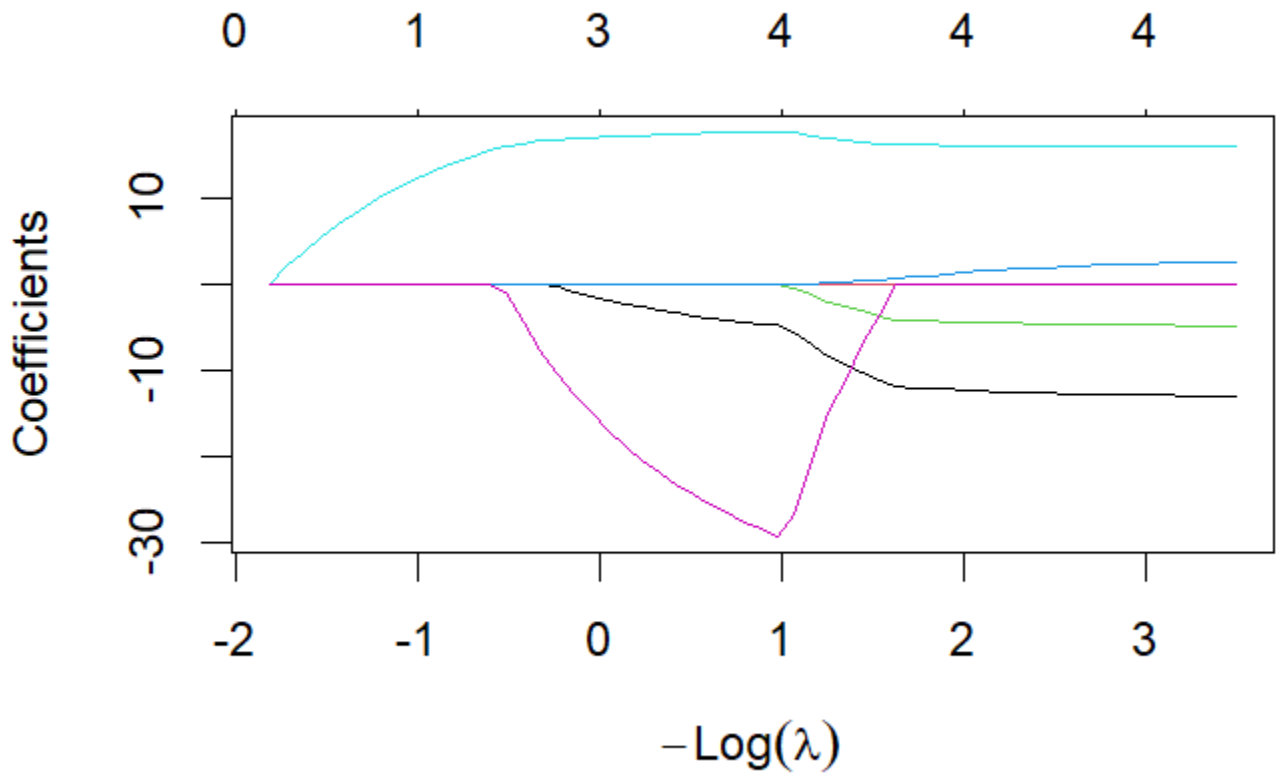
- Ridge conserve toutes les variables dans le modèle, mais réduit la magnitude des coefficients pour limiter le surajustement.
- Les variables les plus influentes restent x6 (positif fort) et x3 / x7 (négatifs marqués).
- x5 est quasiment neutralisé (≈ -0.14), ce qui montre qu'il n'apporte pas d'information utile.

**Régression Lasso**

Pareil que pour la Ridge sauf que maintenant le paramètre alpha vaut 1 :

```
# Régression Lasso (alpha=1)
m_lasso <- glmnet(X, y, alpha = 1)
# Visualiser les coefficients en fonction de lambda
plot(m_lasso, xvar = "lambda")
```

La visualisation des coefficients en fonction de Lambda :



**À gauche (λ grand)** : les coefficients sont fortement contraints et tendent vers zéro → le modèle est très régularisé.

**À droite (λ petit)** : les coefficients reprennent des valeurs plus proches de celles du modèle complet (non régularisé).

Contrairement à Ridge, **certains coefficients deviennent exactement nuls** → c'est la sélection de variables propre au Lasso.

On voit donc que certaines variables « disparaissent » (leurs courbes s'écrasent sur zéro), tandis que d'autres restent actives avec des valeurs significatives.

Le choix du paramètre lambda (validation croisée), le calcul de RMSEP et l'extraction des coefficients du modèle Lasso pour voir l'impact de chaque variable :

```
> cv_lasso <- cv.glmnet(X, y, alpha = 1)
Message d'avis :
Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per fold

> # Lambda optimal
> lambda_opt_lasso <- cv_lasso$lambda.min
> lambda_opt_lasso
[1] 0.08517641
> # RMSEP associé
> rmsep_lasso <- sqrt(mean((y - predict(cv_lasso, s=lambda_opt_lasso, newx=X))^2))
> rmsep_lasso
[1] 0.5731319
> coef_lasso <- coef(cv_lasso, s = "lambda.min")
> coef_lasso
8 x 1 sparse Matrix of class "dgCMatrix"
            lambda.min
(Intercept)  85.649329
x1          -12.611123
x2            .
x3            .
x4           -4.619701
x5            1.908944
x6           15.947236
x7            .
```

λ optimal ≈ 0.085.

RMSEP **Lasso ≈ 0.57**, encore meilleur que Ridge (≈0.68), PCR (≈1.31) et PLS (≈1.30).

Le Lasso a effectué une **sélection de variables** : seules **x1, x4, x5, x6** sont retenues.

Les variables **x2, x3, x7** sont jugées non pertinentes et mises à zéro.

Les variables les plus influentes sont **x6 (positif fort)** et **x1 (négatif marqué)**.

## Comparaison des méthodes de régression (PCR, PLS, Ridge et Lasso)

Dans cette étude comparative de méthodes de régression, plusieurs approches ont été testées afin de prédire l'indice d'octane. La régression par composantes principales (PCR) avec 4 composantes a donné un RMSEP d'environ 1,31, tandis que la régression PLS avec 3 composantes a légèrement amélioré la performance (RMSEP ≈ 1,30) en mettant en évidence l'importance des variables x6 et x4. La régularisation Ridge, avec un $\lambda$ optimal proche de 8, a permis de conserver toutes les variables tout en réduisant leurs amplitudes, et a fortement amélioré la précision (RMSEP ≈ 0,68). Enfin, la régression Lasso, avec un $\lambda$ optimal ≈ 0,085, a obtenu le meilleur compromis entre performance et simplicité, en sélectionnant seulement quatre variables (x1, x4, x5 et x6) et en atteignant un RMSEP ≈ 0,57. Ainsi, le Lasso apparaît comme le modèle le plus performant et le plus parcimonieux, suivi par Ridge, tandis que PCR et PLS restent utiles pour l'interprétation et la réduction dimensionnelle mais moins efficaces en prédiction.